VoltEdge 2.0 — Model and Data Card

This section documents the AI model/system behavior and the data handled by VoltEdge. VoltEdge does not train a new foundation model or a custom machine-learning model; it integrates an external agent model through the Claude Agent SDK and connects that model to a guarded tscircuit workspace.

Model Card

Field	Description
System name	VoltEdge 2.0 Agentic Circuit Design Assistant
Model type	Application-level AI assistant using the Claude Agent SDK with a tscircuit toolchain.
Training status	VoltEdge does not train or fine-tune a new model. The underlying Claude model is configured through backend settings.
Primary purpose	Convert natural-language circuit design requests into editable tscircuit projects with streamed progress and visual schematic/PCB previews.
Intended users	Students, makers, developers, and engineers prototyping simple circuit boards.
Inputs	User prompts, project workspace files, tscircuit/component library files, imported component metadata, and optional UI placement edits.
Outputs	tscircuit source such as `index.circuit.tsx`, build artifacts such as `circuit.json`, assistant messages, tool events, checkpoints, previews, and planned fabrication exports.
Deployment context	Local web application with a FastAPI backend, React frontend, Claude Agent SDK runtime, and tscircuit CLI/toolchain.

The model-facing agent is configured to work inside a project-scoped workspace. It is instructed to build exactly what the user asks for, prefer ready-made library components when available, keep layouts compact, run only necessary checks, and write the circuit to `index.circuit.tsx`, which is the file rendered by the UI.

Implemented safeguards include:

- One active agent turn per project.
- Project-scoped working directory.
- File Write/Edit confinement to the project workspace.
- Bash command allowlist rather than unrestricted shell access.
- Build-artifact checkpointing from generated `circuit.json` updates.
- Human review requirement before fabrication or real-world use.

Known limitations include:

- VoltEdge does not guarantee electrical correctness, safety, or manufacturability.
- The agent may choose incorrect parts, footprints, pin mappings, board dimensions, or routing assumptions.
- Ambiguous natural-language prompts can produce incomplete or incorrect designs.
- The system depends on the behavior and availability of external model and toolchain providers.
- Generated fabrication files must be reviewed by a qualified person before manufacturing.

Out-of-scope uses include safety-critical, medical, automotive, aerospace, high-voltage, or high-current electronics unless a qualified engineer independently reviews and validates the design.

Data Card

VoltEdge does not use a custom training dataset. The data handled by the system is application/session data generated during normal use.

Data category	Examples	Storage/handling
User prompts	Natural-language circuit requirements and follow-up instructions.	Stored as message records and sent to the configured agent provider during an agent turn.
Agent responses	Assistant text, thinking summaries where surfaced, tool-use summaries, and errors.	Streamed over SSE and persisted in event/message history.
Project metadata	Project ID, title, created timestamp, workspace path.	Stored in the local SQLite database.
Workspace files	`index.circuit.tsx`, package metadata, imported component files, tscircuit configuration, and markdown/source files.	Stored in project workspace directories.
Build artifacts	`dist/index/circuit.json`, generated previews, and planned export artifacts such as Gerber/BOM/PnP files.	Stored or read from project workspaces; checkpointed when build artifacts change.
Placement edits	`pcbX`, `pcbY`, `schX`, and `schY` coordinate changes from drag interactions.	Written back into `index.circuit.tsx` through the placement API.
External component metadata	Imported JLCPCB/LCSC part metadata, footprints, pin labels, and related package information.	Stored in generated/imported component files when used.

Privacy and security considerations:

• Users should not enter passwords, API keys, private credentials, or confidential designs unless the deployment environment and model-provider policy are trusted.

• Circuit designs, prompts, and workspace files may contain intellectual property.

• If the configured agent provider is external, prompts and relevant workspace context may be transmitted to that provider according to its API and retention policies.

• Local project/session data is stored in SQLite and filesystem workspaces; access controls depend on the deployment environment.

Quality and bias considerations:

• The system may favor components available in the local parts library, tscircuit registry, or JLCPCB/LCSC import path.

• Less common components or ambiguous module names may be modeled incorrectly.

• Availability of footprints and metadata can affect output quality.

• Generated boards should be validated with schematic review, pinout review, DRC/ERC checks where available, BOM verification, and datasheet inspection.

Recommended validation before fabrication:

• Run `tsci build` successfully.

• Inspect schematic and PCB previews.

• Verify all component pin mappings against datasheets.

• Review board dimensions, clearance, routing, and layer assumptions.

• Check BOM and pick-and-place files.

• Confirm fabrication outputs with the selected manufacturer.